

Docker & AWS Deployment

Command Cheat Sheet — what each command does and when to use it

This is the set of commands we use to take an app from a laptop to the cloud, grouped by stage. Under each command is **what it does and when you would run it**. Placeholders like `<ACCOUNT_ID>`, `<container>` and `<your-key>` mean ‘fill in your own value’.

1 · Install and set up the tools

Done once on your machine, before anything else.

```
brew install awscli aws/tap/copilot-cli gh
```

→ Installs the three CLIs we need: the AWS CLI, AWS Copilot (the cloud deploy tool), and the GitHub CLI. On Windows/Linux, install each from its own docs.

```
aws configure --profile nawaloka
```

→ Points the AWS CLI at your account — access keys, region (us-west-2), output format. The named profile means you never touch the root account.

```
aws sts get-caller-identity --profile nawaloka
```

→ A quick sanity check: prints which AWS identity you are, confirming your keys work and you are not the root user.

2 · Run the app locally with Docker Compose

Rehearse the exact production setup on your own machine — for free — before touching the cloud.

```
docker compose -f compose.prod.yml up
```

→ Starts the whole stack at once — two API copies, the worker, the voice service, Redis, and the web UI — streaming their logs to your terminal.

```
docker compose -f compose.prod.yml up -d
```

→ Same, but detached (in the background) so you get your terminal back.

```
docker compose -f compose.prod.yml down
```

→ Stops and removes the containers when you are done. Your saved data volumes are kept, so next time your data is still there. Do not add `-v` — that also deletes the data volumes.

3 · Look inside the running containers

Everything here is read-only — it just shows you what the containers are doing.

```
docker compose -f compose.prod.yml ps
```

→ Lists your running services with their status and ports — the friendly view of the stack.

```
docker ps
```

→ The same containers one level lower: raw container IDs, the image each came from, uptime, and port mappings.

```
docker compose -f compose.prod.yml logs -f api
```

→ Tails one service’s live logs as it runs (`-f` means ‘follow’). Press Ctrl+C to stop watching — it does not stop the container.

```
docker stats
```

→ A live dashboard of CPU and memory for every container, updating each second. Press Ctrl+C to quit.

```
docker exec -it <container> sh
```

→ Opens a shell inside a running container so you can look around (run `ls`, `env`, `python --version`, then `exit`). Get the name from `docker ps`.

```
docker inspect <container>
```

→ Prints everything Docker knows about a container as JSON — the image, environment variables, mounted volumes, and network.

```
docker images
```

→ Lists the local image 'templates' on your machine that the containers were built from.

4 · Build the image and push it to AWS (ECR)

ECR is your private image registry on AWS — you build the box, then upload it there.

```
aws ecr get-login-password | docker login --username AWS --password-stdin $REGISTRY
```

→ Logs Docker into your private AWS image registry so it is allowed to push images.

```
docker build --platform linux/arm64 -f docker/api/Dockerfile -t  
$REGISTRY/nawaloka/api:latest .
```

→ Builds the application image for arm64 — Amazon's Graviton chips, cheaper for the same performance.

```
docker push $REGISTRY/nawaloka/api:latest
```

→ Uploads the freshly built image to ECR, ready for AWS to run.

5 · Provision and deploy with Copilot

Copilot is 'docker-compose for the cloud': you describe what you want and it builds the AWS plumbing.

```
copilot app init nawaloka
```

→ Creates the Copilot application — the top-level container for everything we deploy. Done once.

```
copilot env init --name dev
```

→ Creates the 'dev' environment, with its own network and cluster. Done once per environment.

```
copilot svc init --name api --svc-type "Load Balanced Web Service" --port 8000
```

→ Registers the API as a load-balanced web service, so it gets a public load balancer. worker and voice are 'Backend Service' — no public door.

```
copilot env deploy --name dev
```

→ Builds the actual cloud infrastructure for the environment: the private network, the load balancer, and the ECS cluster.

```
copilot svc deploy --name api --env dev
```

→ Deploys a service to AWS and starts it running on Fargate. Repeat for worker and voice.

```
copilot svc deploy --name api --env dev --force
```

→ Forces a fresh deploy of the same :latest image — use this when you rebuilt the image but the tag name did not change.

6 · Secrets the right way

API keys and passwords never go in code or images — they live in AWS's encrypted vault.

```
copilot secret init --cli-input-yaml secrets.yml
```

→ Stores each secret encrypted in AWS Parameter Store and automatically lets the running container read it at startup. You reference secrets by name, never by value.

7 · CI/CD — push to deploy

Once set up, you never deploy by hand — pushing your code does it for you.

```
git push origin dev
```

→ Pushing to the dev branch triggers GitHub Actions to build the image, push it to ECR, and roll it out to AWS — automatically.

```
gh run watch
```

→ Follows the running GitHub Actions deployment live from your terminal (or watch it in the Actions tab on GitHub).

8 · Cost — pause and tear down

The cloud bills for every hour a thing exists, even idle. The last command of every session is the shut-down.

```
aws ecs update-service --cluster <CL> --service <SVC> --desired-count 0
```

→ Pauses compute between sessions by scaling the running tasks to zero — stops most of the hourly cost without deleting anything.

```
copilot app delete
```

→ Tears the whole deployment down to roughly zero per day when you are done for a while. Code, images, and secrets stay safe in Git, ECR, and SSM; you can rebuild in about 15 minutes.